



COS30018 - Intelligent System

Project: Stock Price Prediction System

Task C.2 - Data Processing - Part 1

Student Name: Ngo Sy Vuong

Student ID: 105551480

Tutor: Nguyen Manh Toan

Tutorial Session: Wednesday 8:00 - 12:00

Semester: May - July 2026

Table of Contents

Content	Page
Cover Page	1
Table of Contents	2
Environment Setup	
1. Environment	3
2. Dependencies	3
3. Deployment	3
Executive Summary	4
Stock Prediction Version 2	
1 - Local Storage and Caching Mechanism	5
2 - Dealing with the NaN issue	6
3 - Advanced Data Splitting Strategy	8
4 - Multi-Feature Isolated Scaling and Scaler Retention	10

Environment Setup

To successfully replicate and run the stock price prediction model, please fulfill the following structural and software dependencies.

1. Environment

The system is built on an isolated virtual environment (venv) to ensure dependency abstraction and eliminate version conflicts with global system libraries:

- Core Runtime: **Python 3.11.x (64-bit)**
- Execution Interface: **Windows Command Prompt (CMD)**

2. Dependencies

tensorflow	Core Deep Learning framework used to compile structure and execute the LSTM network
yfinance	Automated financial data retrieval tool used to fetch historical structural data for CBA.AX via the Yahoo Finance API
scikit-learn	Provides data preprocessing mechanics (specifically utilized for the MinMaxScaler engine to normalize array bounds between 0 and 1)
pandas	Sequential series concatenation, index cleaning and CSV file extraction
numpy	Low-level vectorized array manipulation, matrix dimension reshaping and numeric tensor optimization
matplotlib	Render visual evaluations

3. Deployment

To reconstruct this environment layout from scratch, run the following sequential commands within the target file directory: **(make sure you have installed Python 3.11)**

```
# Initialize the Python 3.11 Isolation layer  
py -3.11 -m venv venv
```

```
# Activate the localized scripts block (Windows)  
venv\Scripts\activate.bat
```

```
# Synchronize deployment prerequisites  
pip install -r requirements.txt
```

Executive Summary

All architectural constraints specified in the project brief have been programmatically wrapped into a single function:

```
def load_scale_and_split_data(  
    ticker, start, end, features_list, target_col,  
    handle_nan="ffill",  
    split_method="date",  
    split_ratio=0.8,  
    split_date=None,  
    cache_dir="data_cache"  
):
```

Figure 1 - load_scale_and_split_data function

This structural architecture abstracts the complex transformations away from the primary execution script, ensuring complete modularity. The five core operational data components:

- 1. Automated Caching & Header Flattening:**
The pipeline verifies local storage using `os.path.exists()` to bypass unnecessary API network requests. It automatically detects and strips multi-level `pd.MultiIndex` headers from yfinance to prevent data processing crashes
- 2. Missing Data Cleaning:**
It handles incomplete datasets through configurable parameters (Forward-Fill, Backward-Fill, or deletion), protecting the continuous timeline required by LSTM layers
- 3. Flexible Slicing Options:**
Data splitting is fully parameterized, allowing the program to partition datasets based on fixed percentage ratios, specific calendar cutoffs, or randomized shuffling
- 4. Isolated Multi-Feature Scaling:**
Because raw features vary drastically (like prices under \$100 and volume in the millions), the function scales each column independently into a balanced range between 0.0 and 1.0 using `MinMaxScaler`
- 5. Data Leakage Resolution:**
Scaler boundaries are computed (`fit_transform`) solely on training data and applied blindly (`transform`) to test data. These individual configurations are preserved within a `structured scalers = {}` dictionary to allow exact price conversions later.

Stock Prediction Version 2

1 - Local Storage and Caching Mechanism

The data ingestion component handles retrieval efficiency and structural normalization. This phase ensures the application executes rapidly without redundant web dependencies and reformats the incoming data layout so it can be processed by machine learning algorithms without numerical truncation errors.

Downloading historical market data from the web on every script execution introduces network latency, relies on a constant internet connection and risks triggering API rate limits. To establish a more resilient system architecture, the pipeline implements a local file-system caching engine.

```
# 1 - Local Data Storage and Caching Mechanism

os.makedirs(cache_dir, exist_ok=True)
cache_file = os.path.join(cache_dir, f"{ticker}_{start}_{end}.csv")

if os.path.exists(cache_file):
    print(f>Loading baseline dataset locally from: {cache_file}")
    raw_data = pd.read_csv(cache_file, index_col=0, parse_dates=True, header=[0, 1]) # Ensure it does not read the first 2 rows

    # Check if column headers are currently built as a multi-layered index structure
    # If YES: Tells Pandas to isolate only the names found on Level 0 (the top deck: Open, High, Low, Close, Volume)
    # and completely discard Level 1 (CBA.AX)
    # If NO: Continue
    if isinstance(raw_data.columns, pd.MultiIndex):
        raw_data.columns = raw_data.columns.get_level_values(0)
else:
    print(f>Local cache missing. Fetching data remotely via yfinance...)
    raw_data = yf.download(ticker, start=start, end=end)
    if raw_data.empty:
        raise ValueError(f>Download failed. No operational records returned for ticker: {ticker}")
    raw_data.to_csv(cache_file)

    if isinstance(raw_data.columns, pd.MultiIndex):
        raw_data.columns = raw_data.columns.get_level_values(0)

df = raw_data[features_list].copy()
```

Figure 2 - Local Storage and Caching Mechanism

The function creates a dedicated directory on the disk using `os.makedirs(cache_dir, exist_ok=True)` and constructs a distinct filename bound to the specific stock ticker and input date parameters. It then runs a structural validation check using `os.path.exists(cache_file)` to verify if the historical records already reside locally. If the file is detected, the pipeline short-circuits the retrieval loop, bypassing the internet entirely to load the CSV directly into memory from the local drive. If the cache file is missing, the pipeline falls back to the internet, requests a fresh data block via `yf.download()` and immediately writes it to disk to establish a localized data asset for all subsequent runs.

Beyond physical storage, the pipeline must also resolve a structural column layout incompatibility introduced by modern iterations of the Yahoo Finance data engine. When downloading multivariable market data, yfinance returns a multi-tiered array header known as a `pd.MultiIndex`. This formatting organizes column labels into a hierarchical, double-decker framework where the top layer (Level 0) contains the core financial indicator names, and the bottom layer (Level 1) repeats the specific text ticker symbol underneath every single attribute column. When Pandas commits this multi-level dataframe to a local CSV file, it writes two explicit string text rows at the peak of the document before beginning the data rows.

Upon reloading this cached CSV via `pd.read_csv()`, Pandas parses both header layers, successfully reconstructing the multi-index column matrix. However, downstream Scikit-Learn preprocessing tools such as the `MinMaxScaler` are mathematically incapable of handling a multi-layered matrix layout. These packages strictly expect a flat, single-row vector array of feature names. When passed raw multi-index headers, the mathematical transform engine hits the text string layer, fails to convert the values to floating-point metrics and triggers a fatal system crash (`ValueError: could not convert string to float`). To insulate the system against this structural error, the pipeline introduces an automated header flattening block using defensive type-checking:

```
# Check if column headers are currently built as a multi-layered index structure
# If YES: Tells Pandas to isolate only the names found on Level 0 (the top deck: Open, High, Low, Close, Volume)
#           and completely discard Level 1 (CBA.AX)
# If NO: Continue
if isinstance(raw_data.columns, pd.MultiIndex):
    raw_data.columns = raw_data.columns.get_level_values(0)
```

Figure 3 - Header flattening block

The conditional statement verifies whether the loaded columns are formatted as a multi-index structure. If confirmed, `.get_level_values(0)` isolates the primary horizontal layer containing the essential indicator names (Open, High, Low, Close, Volume) while discarding the redundant bottom text strings. This operation flattens the table back into a standard single-header schema, allowing the numerical features to flow into the processing layers without numerical data contamination.

2 - Deal with the NaN Issue

Financial time-series data frequently suffers from missing entries, logged as NaN (Not a Number) values. These gaps are typically caused by public market holidays, truncated half-day trading sessions, structural server transmission dropouts, or temporary asset liquidity pauses. In a deep learning workflow, passing an NaN value into an LSTM network breaks the sequential matrix multiplication inside the recurrent cells, causing the error to cascade until the model outputs nothing but NaN predictions. To insulate the network from these mathematical failures while preserving the structural continuity of the timeline, the pipeline integrates a parameter-driven remediation engine that allows the configuration of three distinct data-cleaning strategies via the `handle_nan` argument.

```
# 2 - Deal with the NaN Issue

if handle_nan == "ffill":
    df = df.fffll()
# ffill = Foward Fill
# Example use:
# Before ffill: [ 62.5, 62.8, NaN, NaN, 63.1 ]
# After ffill: [ 62.5, 62.8, 62.8, 62.8, 63.1 ]

elif handle_nan == "bfill":
    df = df.bfill()
# bfill = Backward Fill
# Example use:
# Before bfill: [ 62.5, 62.8, NaN, NaN, 63.1 ]
# After bfill: [ 62.5, 62.8, 63.1, 63.1, 63.1 ]

elif handle_nan == "drop":
    df = df.dropna()
# Instead of guessing or copying values, dropna() completely
# destroys the entire row containing an $NaN$ entry

if df.isnull().values.any():
    df = df.dropna()
```

Figure 4 - Dealing with the NaN issue

The primary and most robust strategy for financial time-series forecasting is Forward-Fill (ffill). This method handles an empty cell by propagating the last known valid historical value forward to fill the chronological gap. From a financial engineering perspective, this operates on the rational assumption that if a market transaction was not recorded on a specific day (such as a weekend or public holiday), the asset's structural valuation remains equal to its last confirmed closing transaction price. This approach allows the pipeline to maintain an uninterrupted, daily sequential chain without fabricating artificial market movements or introducing forward-looking bias.

Conversely, the pipeline provides a Backward-Fill (bfill) configuration, which executes the exact opposite operation by looking forward in time and copying the next available valid market price backward into the empty slot. While mathematically clean for static tabular datasets, backward-filling introduces a severe flaw known as Look-Ahead Bias (Data Leakage) when applied to time-series forecasting. By allowing future prices to populate past missing states, the training dataset inadvertently leaks future trends to the LSTM model before they chronologically occur, resulting in an artificially inflated training accuracy that fails catastrophically upon real-world deployment.

The final programmatic alternative is Row Deletion (dropna), which completely purges any row containing a NaN entry from the DataFrame. While this ensures that only authentic, unmanipulated market transactions remain, it is highly problematic for Recurrent Neural Networks. LSTMs rely heavily on a fixed, unbroken look-back window (configured via PREDICTION_DAYS = 60) to map temporal trends and momentum. Deleting rows rips physical holes in the timeline, forcing the model to read two disjointed dates as if they occurred on consecutive days, which distorts the structural patterns the network is trying to learn.

To handle edge cases where these methods might leave remaining gaps, the pipeline concludes with a defensive diagnostic check (See the 2 final lines in Figure 4). This safety conditional scans the compiled feature matrix for any lingering NaN values. This step is vital because if a dataset contains an NaN value at the very first row of its time series, a forward-fill operation has no historical boundary anchor to copy forward, leaving the cell blank. This final block detects and purges those rare edge cases, guaranteeing that the multi-feature matrix passed into the downstream scaling and tensor-generation layers is completely numeric and structurally stable.

3 - Advanced Data Splitting Strategy

Evaluating the real-world performance of a predictive model requires partitioning the source dataset into a Training Set to teach the network historical patterns, and an independent Testing Set to serve as an unseen final exam. Because stock prices are highly dependent on chronological order, the method chosen to divide these sets can fundamentally change how the model learns. The pipeline parameterizes this choice through the split_method argument, supporting three interchangeable operational frameworks.

The first approach is Ratio-Based Splitting (split_method == "ratio"), which divides the data based on a fixed mathematical proportion controlled by split_ratio (like 80% training and 20% testing). The pipeline calculates the exact row count representing the training threshold (split_idx = int(len(df) * split_ratio)) and utilizes integer location-based slicing (iloc) to cut the dataset. Rows from index 0 up to split_idx are allocated to the training DataFrame, while all remaining subsequent rows are pushed to the testing DataFrame. This preserves the absolute sequential flow of time, using the oldest data for learning and the most recent data for validation.

```
# 3 - Advanced Data Splitting Strategy

train_df = pd.DataFrame()
test_df = pd.DataFrame()

if split_method == "ratio":
    split_idx = int(len(df) * split_ratio)
    train_df = df.iloc[:split_idx].copy()
    test_df = df.iloc[split_idx:].copy()
# This method splits the data based on a pure mathematical proportion
# (like 80% training and 20% testing)
# 1/ Calculates how many rows represent the training chunk
#   If there are 1,000 trading days and a ratio of 0.8, it calculates 800
#   Then converts that number into a clean index integer (split_idx = 800)
# 2/ Uses integer-location slicing (iloc) to grab all rows from row 0 up to row 799 for the training set
# 3/ Grabs everything from row 800 to the very end for the testing set
```

Figure 5 - Ratio-Based Splitting

The second framework is Date-Based Splitting (`split_method == "date"`), which replaces row calculations with an explicit calendar date boundary line using label-location slicing (`loc`). The pipeline runs a defensive check to ensure a valid `split_date` string (such as `'2023-08-02'`) is provided, throwing a `ValueError` if it is missing. It then routes all trading records dated strictly before the cutoff into `train_df`, and all records landing on or after that boundary into `test_df`. This method is the standard for financial backtesting because it perfectly simulates a production deployment, allowing the engineer to test how a model trained on past data would perform from a known chronological point forward.

```
elif split_method == "date":
    if not split_date:
        raise ValueError("A valid 'split_date' must be specified when using split_method='date'.")
    train_df = df.loc[df.index < split_date].copy()
    test_df = df.loc[df.index >= split_date].copy()
# Instead of using a row count, this method uses an explicit calendar date
# as a hard boundary wall to separate the past from the future
# 1/ Checks if passed a valid date string (like '2023-08-02')
#   If NO, throw a ValueError crash to prevent errors
# 2/ Uses label-location slicing (loc) to scan the DataFrame's index (which contains the Date timestamps)
#   and groups any trading day happening before that date into train_df
# 3/ Groups any trading day landing on or after that cutoff date into test_df
```

Figure 6 - Date-Based Splitting

The final operational alternative is Randomized Splitting (`split_method == "random"`), which bypasses temporal boundaries entirely. It leverages Scikit-Learn's `train_test_split()` helper function to scramble the daily records into a randomized distribution based on a fixed `random_state=42` seed for calculation reproducibility. While highly effective for static tabular datasets (such as classifying images or customer profiles), randomized splitting is heavily flawed when applied to deep learning time-series models.

```
elif split_method == "random":
    train_df, test_df = train_test_split(df, train_size=split_ratio, shuffle=True, random_state=42)
# 1/ Calls Scikit-Learn's built-in train_test_split() helper function
# 2/ shuffle=True tells the function to shuffle the entire timeline randomly
# 3/ train_size=split_ratio ensures that despite the random shuffling,
#   train_df still gets 80% of the total available rows and test_df gets the remaining 20%
# 4/ random_state=42 is a pseudo-random seed. It ensures that the "random" shuffle happens
#   exactly the same way every time running, making the results reproducible
```

Figure 7 - Randomized Splitting

Shuffling data breaks the continuous, day-to-day chronological sequence that Recurrent Neural Networks rely on to capture multi-day momentum and trend patterns. Furthermore, it introduces severe Data Leakage, as the training set ends up containing random days scattered right next to testing days, giving the model an unrealistic, unearned advantage during training that fails completely when deployed on actual future market movements.

4 - Multi-Feature Isolated Scaling and Scaler Retention

Financial datasets track market variables with drastically different numerical scales and magnitudes. While standard stock prices typically sit in double or triple digits (like \$65.00), daily trading volume metrics frequently scale into millions of units. If these raw features are fed directly into a deep learning neural network, the massive numeric variance of the volume data will disproportionately dominate the gradient descent weight optimization, causing the network to ignore critical micro-trends in the price columns. To establish balanced feature weights, the pipeline isolates each metric and compresses its values into a uniform decimal boundary between 0.0 (absolute minimum) and 1.0 (absolute maximum) using individual Scikit-Learn MinMaxScaler objects.

```
# 4 - Multi-Feature Isolated Scaling and Scaler Retention

# 1/ Instead of scaling the whole table at once, the code loops through the features
#   (Open, High, Low, Close, Volume) one column at a time.
#   This is vital because price columns sit around double digits (like $63.00),
#   while Volume sits in the millions (like 1,416,232).
#   Slicing them column-by-column allows normalizing them independently
# 2/ scaler = MinMaxScaler(feature_range=(0, 1)) scales the data
#   into the range between 0.0 and 1.0
# 3/ Train:
#   - fit: The scaler scans the training column to find its
#         absolute minimum (X_min) and maximum (X_max) values.
#   - transform: It applies the math formula to every row in that training column,
#               turning the raw dollar values into decimals between 0 and 1
#   - reshape(-1, 1) and flatten(): Scikit-Learn's scaler strictly demands a 2D matrix array as an input,
#                                   so reshape(-1,1) bumps a flat list into a vertical column matrix.
#                                   Once the math is done, .flatten() squashes it back into a simple 1D list
#                                   so it can neatly fit back into the Pandas DataFrame
# 4/ Test:
#   Same steps as Train, but by calling only .transform(), the code forces the testing data to be scaled
#   using the exact same minimum and maximum boundaries learned from the training set

scalers = {}
scaled_train = pd.DataFrame(index=train_df.index)
scaled_test = pd.DataFrame(index=test_df.index)

for col in features_list:
    scaler = MinMaxScaler(feature_range=(0, 1))

    # Train
    train_matrix = train_df[col].values.reshape(-1, 1)
    scaled_train[col] = scaler.fit_transform(train_matrix).flatten()

    # Test
    test_matrix = test_df[col].values.reshape(-1, 1)
    scaled_test[col] = scaler.transform(test_matrix).flatten()

    scalers[col] = scaler
```

Figure 8 - Multi-Feature Isolated Scaling and Scaler Retention

The pipeline loops through the designated features array column-by-column rather than scaling the entire matrix at once. This independent handling is vital because each market indicator represents an entirely different physical metric. For every column, a dedicated `MinMaxScaler` instance is initialized. The normalization process executes across two distinct mathematical phases to completely insulate the pipeline against Data Leakage:

- **The Training Phase (`fit_transform`):** The scaler scans the training subset (`train_df`) to discover its independent absolute minimum (`X_min`) and maximum (`X_max`) boundaries. It then applies the transformation formula to every row in that specific training column, converting the nominal dollar values into localized decimals between 0.0 and 1.0. The input array is dynamically augmented using `.reshape(-1, 1)` to meet Scikit-Learn's 2D data demands, and subsequently squashed back into a standard 1D array list via `.flatten()` to match the destination Pandas DataFrame structure.
- **The Testing Phase (`transform`):** To preserve strict experimental integrity, the pipeline forces the independent testing subset (`test_df`) to be scaled using the exact same maximum and minimum boundary parameters previously calculated from the training data. By calling a transform-only command (`.transform()`), the testing data is prevented from altering the scaling framework. If the stock breaks out to an unprecedented historical high during the test period, the value naturally scales to a decimal greater than 1.0 (like 1.2), allowing the LSTM layers to recognize a market breakout honestly without peeking ahead.

At the conclusion of each loop iteration, the fully trained column transformer is stored inside a structured dictionary memory layout (`scalers = {}`), mapped directly to its feature column name key. Retaining these standalone objects is critical for downstream operational evaluation. Because the LSTM network outputs its final price predictions as standardized decimals between 0.0 and 1.0, the script must be able to recall the precise, isolated parameters of the target price column later. This retention mechanism allows the visualization module to execute an exact inverse mathematical transformation (`inverse_transform`), translating the model's abstract decimal calculations back into nominal, human-readable asset prices.